Food Detection & Portion Estimation

Two questions from one camera frame — what food, and how much — entirely on a $30 microcontroller.

---

5 + 3
classes + portion tiers

216.7 KB
int8 model on device

1.08 s
on-device inference

0.643
macro-F1, int8 test set

Luke Valerio · Daniel Yang EE 446 TinyML · Summer 2026 · University of Washington

Arduino Nano 33 BLE Sense (nRF52840) · OV7675 camera · TensorFlow Lite Micro, int8

Problem, inputs, and the end-to-end pipeline

one 96×96 frame in, two predictions out — no network, no host

OV7675	Crop	Downsample	Quantise	CNN	Two heads	Output
176×144 RGB565	144×144 centre square	96×96×3 2×2 box mean	int8 scale 1/255	157,512 parameters	5 classes 3 tiers	Serial · LED dashboard

Task

Single plate, single food.

Head A 5-way classification

Head B 3-tier ordinal portion

Tiers map to mass: <80 g · 80–180 g · >180 g.

Sensor

OV7675 on the Tiny ML shield.

QCIF 176×144 RGB565 at 5 fps.

Downsampled, never centre-cropped — the portion tier is measured against the plate rim, so the whole plate must stay in frame.

Evaluation metrics

Class macro-F1 (primary), accuracy

Portion accuracy, mean ordinal error, off-by-two rate

Macro-F1 because per-class confusability, not frequency, is the failure mode.

Everything runs on the device. No companion phone, no cloud inference, no network — the board captures, preprocesses, infers and reports on its own.

Data: real photographed food, honestly labelled

FoodSeg103 (Apache-2.0) mask cutouts composited onto a measured plate

our class	FoodSeg103 source	cutouts	caveat
broccoli	broccoli (87)	986	clean
rice	rice (66)	662	clean
potato	potato (70)	1,073	french fries excluded
chicken	chicken duck (48)	1,226	merged label
beef	steak (46)	1,049	substitution — no beef class

Portion labels are constructed, not estimated

Food is pasted at a sampled fraction f of plate area, so the label is exact by definition: mass ≈ f × 513 g (22 cm plate, 1.5 cm deep, 0.9 g/cm³).

Plate apparent diameter is jittered to 62–92% of frame width, so raw pixel area is an ambiguous cue and the model must measure against the rim.

Corpus

4,996 RGBA mask cutouts

18,000 composited plates

Tiers balanced 5,054 / 5,051 / 4,895.

Split 15,000 train / 3,000 test.

Preprocessing

Resize to 96×96, scale to [0,1].

Augmentation: geometry + sensor degradation only.

No photometric jitter — these are real photographs that already carry real scene lighting; stacking jitter on top crushed batches to black and destroyed the colour cue.

Model: one backbone, two heads that pool differently

depthwise-separable CNN, 157,512 parameters, 13.6 M MACs per frame

96²×3 → stem s2 → 48²×16
→ b1–b2 → 24²×32
→ b3–b4 → 12²×64
→ b5–b6 → 6²×128
→ 1×1 widen → 6²×192
├── MAX pool 6×6 → 5 classes (965 params)
└── AVG pool 6×6 → 3 tiers (579 params)

6 inverted-residual blocks · 3 residual adds · 155,968 parameters shared between the heads. Deployed ops: CONV_2D ×13, DEPTHWISE_CONV_2D ×6, ADD ×3, MAX_POOL_2D, AVERAGE_POOL_2D, FULLY_CONNECTED ×2, SOFTMAX ×2.

Design lineage — where this sits against the YOLO 'nano' family

	params	compute	deployed size	target
YOLO Nano (2018)	—	4.57 B ops	~4.0 MB	Jetson-class
YOLOv8n	3.2 M	8.7 B FLOPs	~6 MB	mobile GPU / CPU
ours	**157.5 K**	**27.2 M FLOPs**	**216.7 KB**	**Cortex-M4 @ 64 MHz**

Kept: depthwise-separable convolutions and residual blocks — the same efficiency toolkit. **Dropped:** anchors, box regression, objectness, NMS and the multi-scale neck. Not for accuracy — for RAM. YOLOv8n's input tensor alone at 640² is 1.2 MB, 4.7× this chip's entire SRAM.

Why the heads pool differently

Food covers a minority of the frame.

Class head max-pools — presence. A max survives a mostly-empty frame.

Portion head average-pools — extent. An average measures how much.

One shared global-average pool stalled the classifier at ~0.34 validation accuracy.

Export decisions

Sized pooling, not Global* — exports better-tested TFLM kernels.

Batch-1 clone on export; a None batch emits dynamic-shape ops.

Outputs bound by width (5 / 3), never by index.

Approaches evaluated, and our advanced components

four training configurations, all scored on the same 3,000-image real test set

training configuration	macro-F1	class acc	portion acc
synthetic corpus only	0.5074	0.5403	0.8290
real corpus, unweighted	0.5859	0.6393	0.9233
real + synthetic combined	0.5330	0.5957	0.8770
real corpus + class weights	**0.6462**	**0.6460**	**0.9237**

Mixing corpora is worse than real alone — replace the synthetic data, do not augment with it.

Class weights are required — without them chicken collapses to F1 0.115 / recall 0.063 on a perfectly balanced corpus. That is confusability, not frequency.

Advanced component 1 — multiple approaches compared

Four corpora/weighting configurations, plus a head-pooling ablation, each trained and scored end to end on an identical test set.

The corpus choice moved macro-F1 by 0.139 — more than any architecture change we tried.

Advanced component 2 — output beyond the Serial Monitor

On-board RGB LED signals capture, result and camera fault without a host attached.

A live host dashboard (harness/camera_view.py) streams the frame the model actually sees, with a plate-framing guide ring and the prediction overlaid.

Baseline model performance — before compression

float32, 3,000-image held-out test set · no compression applied yet

split	macro-F1	class acc	portion acc	ordinal err
train	0.6832	0.6850	0.9261	—
validation	0.6475	0.6376	0.9253	—
test	**0.6462**	**0.6460**	**0.9237**	**0.0763**

Train-to-test gap 0.037 — the model is not overfitting. The ceiling is set by the source data, not by capacity.

Per-class F1, float32 test set

broccoli	beef	rice	potato	chicken
0.969	0.665	0.567	0.522	0.508

Portion head

0.9237 tier accuracy

0 of 3,000 off-by-two errors — when it is wrong, it is wrong by exactly one tier.

Mean ordinal error 0.0763.

Where the class errors are

Rice ↔ potato is the dominant confusion: 183 rice→potato and 126 potato→rice.

Both are pale, starchy and near-textureless at 96×96. Broccoli, which is unmistakably green, reaches 0.969.

Compression: int8 post-training quantization

2.8× smaller for 0.003 macro-F1 — the whole TinyML trade in one row

model	macro-F1	portion acc	model size	tensor arena	flash	latency
float32 (baseline)	0.6462	0.9237	621,956 B	—	—	does not fit
int8 PTQ (deployed)	**0.6432**	**0.9187**	**221,888 B**	**113,516 B**	**500,648 B**	**1,082 ms**

2.80×
compression ratio

−0.0030
macro-F1 cost

43.3%
of SRAM, arena alone

50.9%
of 983 KB flash

1,082 ms
mean of 20 runs

Quantization settings

Full-integer post-training quantization; int8 weights and activations, int8 input and output.

Representative dataset: 300 training images with augmentation ON — the quantiser should see the degraded activations the device will actually produce.

Measured RAM budget, on hardware

Fixed overhead (mbed OS + TFLM + Serial) 51,792 B

RGB565 QCIF frame buffer 50,688 B

Tensor arena in use 113,516 B

Total static RAM 222,688 B of 262,144 B

Largest free block still allocatable 54,284 B

On-device evaluation — live camera, labelled captures

real plates photographed by the OV7675 · sample counts stated explicitly

class	# on-device samples	correct	recall	F1
beef	23	17	0.739	0.850
chicken	20	6	0.300	0.387
broccoli	19	15	0.789	0.833
rice	13	12	0.923	0.600
potato	0	—	food not available	
overall	**75**	**50**	**0.667**	**0.668**

75 labelled captures with harness/live_capture.py. Macro-F1 is over the four classes with on-device support; potato was not available and is reported as an explicit zero.

Our chicken and beef are raw; the training food is cooked. We predicted the effect before capturing, and it cut both ways.

Pale raw chicken collides with rice — 11 of 20 chicken captures called rice, 3 potato. **Raw beef goes the other way** — the measured failure was that the camera renders cooked beef neutral grey (R−B +1.4 against +39.8 in training). Raw beef is strongly red, and recall went from 0/5 to 17/23 at precision 1.000.

Capture-to-result latency, measured

Capture (bit-banged readFrame) 331–583 ms

Preprocess (crop, downsample, quantise) ~68 ms

Inference 1,082 ms

Total 1,482–1,733 ms

Portion head on device — a degenerate output

The head emitted 'large' on all 75 captures, every one at exactly 0.9961 = 255/256, the int8 softmax ceiling. It never once produced 'small' or 'medium'.

Apparent accuracy 0.478 (33/69 labelled) is the base rate alone — 33 of the plates happened to be large. Recall 1.000 on large, 0.000 on small and medium.

Class head verified against the host: img_sum = input_sum = 449,425 · match = OK

Key findings

what moved the numbers, and what did not

Data beat architecture, decisively

Switching from synthetic textures to real FoodSeg103 cutouts, plus class weighting, moved macro-F1 from 0.5074 to 0.6462.

+0.139 from data alone, at identical model size, arena and latency.

int8 was nearly free

2.80× smaller for 0.0030 macro-F1 and 0.005 portion accuracy.

The portion head lost nothing structurally: still zero off-by-two errors in 3,000 test images after quantization.

The bottleneck is not the kernels

CMSIS-NN is already active — 26 int8 symbols, 251 DSP SIMD instructions — and measured no speedup at all (1,092 ms vs 1,076 ms reference).

1,082 ms remains unexplained.

Challenge — the device disagreed with the host

A byte-identical input produced a different class on device. Three hypotheses were wrong before an on-device checksum isolated it: Invoke() aliases and destroys its own input tensor.

Fix: refill the input before every Invoke().

Challenge — the portion head pins at 'large' on camera input

Diagnosed on the host, not guessed. Ruled out: white balance, contrast, and mean/std matching all leave it pinned.

A scale sweep shows the head is correct from 0.45× to 1.35× plate zoom, and only fails once the rim leaves the frame.

Limitations and what we would do next

the honest boundary of what these numbers support

Immovable without new data

FoodSeg103 has no beef and no pure chicken. Our beef is steak; our chicken is a merged 'chicken duck' label. This caps chicken near F1 0.51 — training cannot fix it.

Rice and potato are not separable at 96×96. Grain structure sits at the resolution limit, and higher resolution breaks the RAM budget.

The food is real but the plate is drawn. Real shadows, specular highlights and depth are absent from training.

Open and unverified

Portion tiers are modelled, not weighed. 0.9187 is scored against our own definition of a tier; real gram accuracy is unmeasured.

The portion head is degenerate on real camera input. It emitted 'large' on all 75 captures and never produced another tier. Host diagnosis rules out white balance, contrast and mean/std matching; a scale sweep shows the head is correct from 0.45× to 1.35× plate zoom. Unresolved.

Single-label only. The class head ends in softmax, so it reports one food per plate.

Next: multi-label is feasible on this hardware at essentially zero memory cost — 5 independent sigmoids plus a 5×3 per-class tier head takes the output tensor from 8 to 20 values and leaves the arena unchanged. The class head already max-pools, the correct operator for presence detection, so only the activation, loss and data generator change. Beyond that: ~150 weighed captures at a fixed camera height would close the plate-realism and gram-accuracy gaps together.